

CUSTOMER ONBOARDING AI

Master Beginner Walkthrough

From AI-Native Engineering Foundations to the Controlled UI Renderer

This version preserves the canonical learning sequence and includes the Claude Code implementation prompts for Tasks 1–4 alongside the explanations of the artifacts and engineering decisions they produced.

Customer Onboarding AI

Master Beginner Walkthrough — From AI-Native Engineering Foundations to the Controlled UI Renderer

****Purpose:**** This document is the chronological teaching walkthrough for the Customer Onboarding AI project. It connects the earlier conceptual documents with the actual repository implementation work and preserves the sequence in which the ideas and implementation were developed.

How to Use This Walkthrough

Read this document from top to bottom the first time.

The project deliberately does ****not**** begin with React, FastAPI, an AI API, databases, or a complete onboarding workflow. Instead, it establishes the engineering environment first and then adds one small, verifiable capability at a time.

The authoritative implementation artifacts remain in the repository:

- ``CLAUDE.md``
- ``README.md``
- ``docs/``
- ``specs/``
- ``domain/``
- ``ui/``
- ``tests/``
- ``evals/``
- ``.claude/``

This walkthrough explains ****why** each artifact exists, how the pieces fit together, what Claude Code was asked to do, and what was deliberately not built yet******.

****Core principle:**** *AI can propose; deterministic software decides what is allowed to proceed.*

Prompt Record

This walkthrough now preserves the implementation prompts alongside the explanation of the artifacts they produced.

- **Task 1:** Complete verbatim project-foundation prompt.
- **Task 2:** Complete verbatim JSON Schema prompt.

The prompt is part of the teaching material because the project is demonstrating not only *what Claude Code built*, but also *how the engineering task was constrained before Claude Code was allowed to build it*.

1. Canonical Learning Sequence

The teaching sequence is intentionally broader than the numbered implementation tasks.

The documents created during the discussion should be read in this order:

Order • Document • Purpose

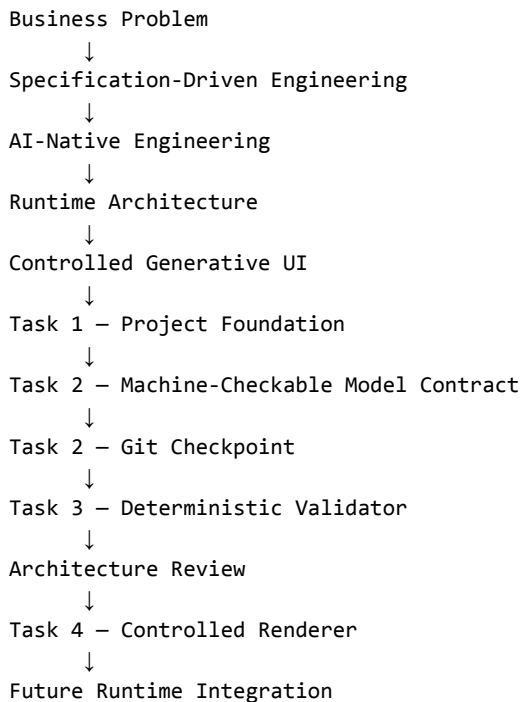
- 1 • **00 — AI-Native Engineering / Claude Code Initial Discussion** • Establishes the business problem, specification-driven engineering, Claude Code, `CLAUDE.md`, and the transition from conventional software engineering to AI-native engineering.
- 2 • **02 — AI-Native Engineering Architecture Beginner Guide** • Explains frontend/backend, UI/Domain/AI runtime views, harness engineering, Claude Code, tests, evals, feedback, and production architecture.
- 3 • **03 — Customer Onboarding Generative UI First Implementation Beginner Guide** • Defines the first small model-driven scenario: controlled Generative UI for the Customer Onboarding landing page.
- 4 • **Customer Onboarding Task 1 Beginner Guide** • Records the first repository task: establish the project engineering foundation.
- 5 • **Customer Onboarding Task 2 Beginner Guide** • Converts the prose UISpec model contract into a machine-checkable JSON Schema.
- 6 • **Customer Onboarding Task 2 Checkpoint Beginner Guide** • Preserves the verified Task 2 state as a Git checkpoint before moving on.
- 7 • **Customer Onboarding Task 3 Beginner Guide — Complete** • Implements the deterministic UISpec validator and its tests.

8 • ****Customer Onboarding Architecture Review Beginner Guide**** • Pauses implementation after Task 3 and reviews the runtime boundaries before the renderer is introduced.

9 • ****Task 4 — Controlled UISpec Renderer**** • Implements the approved component registry and deterministic React renderer.

Why this order matters

The order tells a story:

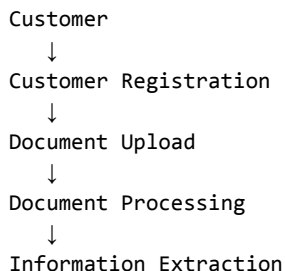


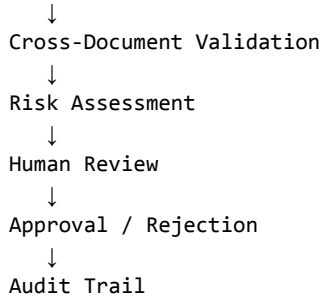
This is intentionally ****not**** simply “Task 1, Task 2, Task 3, Task 4.” The earlier conceptual documents explain why the tasks exist.

2. The Business Problem

The running enterprise example is an internal Virtusa Customer Onboarding application.

A complete onboarding workflow can eventually include:





The long-term system therefore combines:

- ordinary deterministic software,
- business workflows,
- document processing,
- information extraction,
- validation,
- AI-assisted analysis,
- human oversight,
- auditability,
- and controlled decision workflows.

The project is useful as an AI-native engineering example because the system contains both deterministic and probabilistic responsibilities.

The important engineering question is not merely:

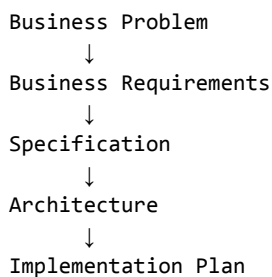
“How do we add an AI model?”

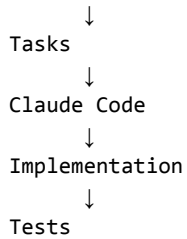
It is:

“How do we engineer the environment around probabilistic behaviour so that the overall system remains controlled, testable, reviewable, and incrementally improvable?”

3. The Transition From Specification-Driven Engineering to AI-Native Engineering

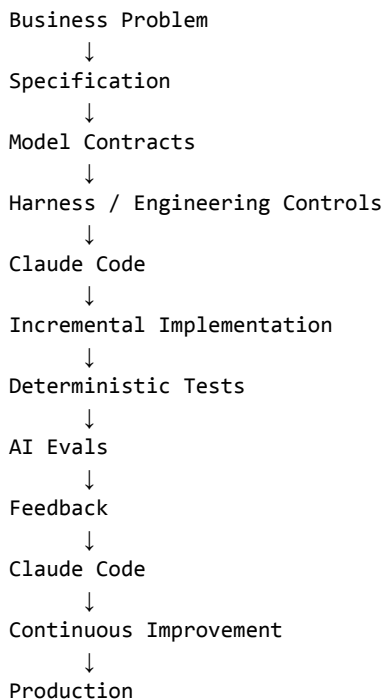
The initial engineering discussion established a conventional sequence:





This is still valid.

AI-native engineering extends the lifecycle because the application itself contains probabilistic model behaviour:

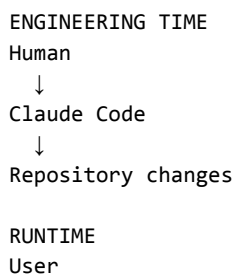


Important distinction

****Claude Code is not the customer-facing AI runtime.****

Claude Code is the engineering-time coding agent used to inspect, modify, test, and reason over the repository.

The eventual Customer Onboarding application may contain separate runtime AI models.



↓
Customer Onboarding Application
↓
Runtime AI Model(s)

Keeping these two roles separate prevents a common conceptual mistake: treating the coding agent and the application's runtime model as if they were the same component.

4. Key Vocabulary

UI Specification / UISpec

A structured JSON document describing the UI that should be rendered.

It is **data**, not executable code.

Model Contract

The written agreement describing what the runtime model may return, which components exist, what properties they accept, and what constitutes invalid output.

The main contract lives in:

``specs/model-contracts.md``

JSON Schema

A machine-checkable representation of the structural part of the model contract.

The v0 schema lives in:

``specs/schemas/ui-spec.schema.json``

Validator

Deterministic software that checks model output against the JSON Schema and additional runtime rules.

The Task 3 implementation lives in:

``domain/validator.py``

Component Registry

A fixed, human-authored mapping between approved component names and actual React components.

Task 4 implements it in:

``ui/src/registry.ts``

Controlled Generative UI

The model proposes a structured UI description. The application validates that description and renders it only through approved components.

Harness Engineering

The broader engineering practice of shaping the environment around an AI agent through project context, rules, tools, validation, tests, feedback, and other mechanisms that make agentic work more reliable.

`CLAUDE.md` is one part of that environment. It is **guidance/context**, not a complete enforcement mechanism.

Deterministic Test

A test whose result should be reproducible for the same input and code state.

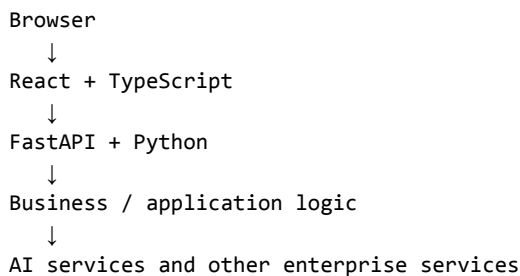
AI Evaluation

A measurement of probabilistic model behaviour across a set of scenarios. Evals complement deterministic software tests; they do not replace them.

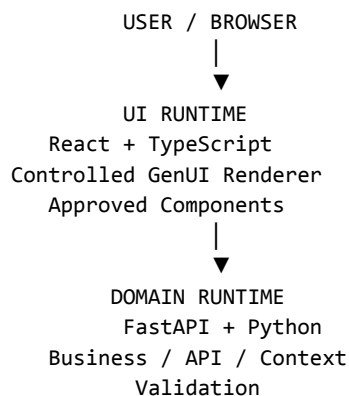
5. Architecture: Frontend / Backend and Runtime Views

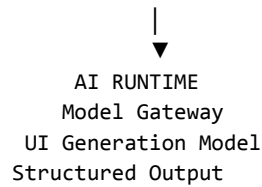
A normal enterprise web architecture still has a frontend and backend.

For this project:



The project also uses a logical runtime view:





These are not competing architectural standards.

- **Frontend/backend** is a practical implementation and deployment view.
- **UI/Domain/AI Runtime** is a responsibility-oriented runtime view.
- **Harness/Claude Code/Tests/Evals/Feedback** describe the engineering environment around the system.

6. The First Implementation Scenario

The first implementation is deliberately small:

Build the Customer Onboarding landing page as a Controlled Generative UI experience.

The landing page is conceptually:

```

Virtusa
Customer Onboarding

Welcome to Customer Onboarding
Start a new customer onboarding journey
or continue an existing application.

[ Customer Setup ]   [ Document Check ]   [ Risk Review ]

      [ Start Customer Onboarding ]
  
```

The purpose is not to demonstrate a complicated visual design.

The purpose is to demonstrate the complete model-driven engineering boundary:

```

AI proposal
  ↓
Structured UISpec
  ↓
Schema validation
  ↓
Deterministic validation
  ↓
Approved UISpec
  ↓
Approved component registry
  ↓
React renderer
  ↓
Browser
  
```

7. Controlled Generative UI

There are two fundamentally different approaches.

Free-form Generative UI

The model could produce arbitrary:

- HTML
- JavaScript
- React
- executable code
- browser instructions

That makes the model part of a privileged execution path.

Controlled Generative UI

The model produces structured data.

For example, conceptually:

```
{
  "version": "v0",
  "root": {
    "component": "Hero",
    "props": {
      "headline": "Welcome to Customer Onboarding"
    }
  }
}
```

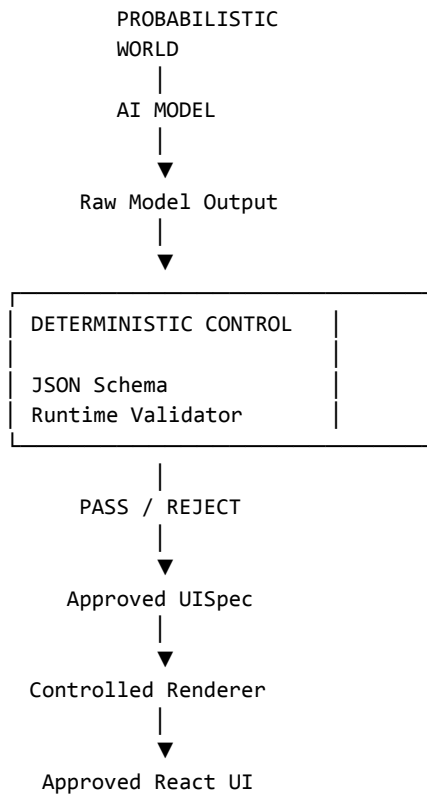
The application then decides:

1. Is this structurally valid?
2. Does it satisfy runtime rules?
3. Is the component approved?
4. Can the approved renderer safely render it?

The model does ****not**** write the React implementation.

8. The Trust Boundary

The central architecture principle is:



The renderer therefore has an important contract:

```
render(approvedUISpec)
```

not:

```
render(rawAIOutput)
```

This distinction is one of the most important lessons in the project.

*****AI can propose. The system decides what is allowed.*****

9. Task 1 — Establish the Project Foundation

Exact Claude Code Prompt Used for Task 1

The following is the complete Task 1 prompt used to establish the project foundation:

We are building an internal Virtusa Customer Onboarding AI application.

This repository is currently empty.

Do NOT build the application yet.

First, establish the project engineering foundation for an AI-native, model-driven

application.

The first implementation scenario is intentionally small:

Build a Customer Onboarding landing page using controlled Generative UI.

The runtime AI model will generate a structured UI specification, not executable React, HTML, JavaScript, or shell commands.

The application will validate the model output and render it only through an approved set of React components.

This is an internal Virtusa application, so do NOT create a separate authentication subsystem for this initial implementation.

Create the following project foundation:

1. CLAUDE.md
2. README.md
3. docs/business-problem.md
4. docs/architecture.md
5. specs/spec.md
6. specs/model-contracts.md
7. tests/
8. evals/
9. .claude/

The documentation should clearly distinguish:

- Business requirements
- System specification
- Model contract
- Claude Code engineering instructions
- Deterministic software tests
- AI evaluations

Use the following architecture concepts:

UI Runtime

- React
- TypeScript
- Controlled Generative UI renderer

Domain Runtime

- FastAPI
- Python
- Initial business/API layer

AI Runtime

- Model gateway
- UI generation model
- Structured model output

The model must NOT generate executable code.

The model may:

- select approved UI components
- arrange approved components
- generate user-facing text
- adapt content to context
- return a valid UI specification

The model must NOT:

- execute commands
- access databases directly
- generate arbitrary executable code
- make customer approval decisions
- invent unavailable capabilities
- bypass the UI component registry

The first approved component vocabulary should include:

Hero
 Heading
 Text
 FeatureCard
 FeatureGrid
 CTA
 Alert
 InfoPanel
 ProgressIndicator

Do not implement document processing, customer registration, validation AI, risk AI, human approval workflow, or production infrastructure yet.

Do not add authentication.

Do not over-engineer the project.

Before creating files, inspect the repository and propose the structure you intend to create.

Then create the foundation.

After creating the files, explain what was created and identify the next smallest implementation task.

Do not proceed to the next task automatically.

9.1 Starting point

The repository was intentionally treated as an empty project foundation.

The first task did ****not**** build the application.

The objective was to create the durable engineering context that Claude Code would use in later tasks.

9.2 Why start with documentation?

The first task answers six different questions:

Layer • Question

Business • Why are we building this?

System • What should the system do?

Model • What may the runtime AI model do?

Engineering • How should Claude Code work on the repository?

Tests • What deterministic behaviour must always be correct?

Evals • How will we measure probabilistic AI behaviour?

Putting all six into one prompt would make the project difficult to maintain.

Instead, each kind of knowledge gets a durable home.

9.3 Task 1 repository foundation

```
customer-onboarding-ai/  
├── CLAUDE.md  
├── README.md  
├── docs/  
│   ├── business-problem.md  
│   └── architecture.md  
├── specs/  
│   ├── spec.md  
│   └── model-contracts.md  
├── .claude/  
│   └── settings.json  
├── tests/  
│   └── README.md  
└── evals/  
    └── README.md
```

`CLAUDE.md`

Persistent project guidance for Claude Code.

It describes:

- project purpose,
- current scope,
- architecture,
- approved vocabulary,
- hard boundaries,
- information ownership,

- engineering conventions.

It is guidance and context, not a hard security boundary.

``README.md``

The human-facing project overview.

``docs/business-problem.md``

The business “why”.

``docs/architecture.md``

The architectural “how”.

``specs/spec.md``

The system “what”.

``specs/model-contracts.md``

The runtime AI contract.

``tests/README.md``

Defines the role of deterministic software tests.

``evals/README.md``

Defines the role of AI evaluations.

``.claude/settings.json``

Minimal project-level Claude Code settings.

No unnecessary hooks, permissions, environment variables, or infrastructure were introduced at this stage.

10. Task 1 — Model Boundaries

The runtime model may:

- select approved UI components,
- arrange approved components,
- generate user-facing text,
- adapt content to context,
- return a valid UI specification.

The runtime model must not:

- execute commands,
- access databases directly,
- generate arbitrary executable code,
- make customer approval decisions,

- invent unavailable capabilities,
- bypass the UI component registry.

The initial approved component vocabulary is:

Hero
 Heading
 Text
 FeatureCard
 FeatureGrid
 CTA
 Alert
 InfoPanel
 ProgressIndicator

The first task deliberately excluded:

- document processing,
- customer registration,
- validation AI,
- risk AI,
- human approval workflow,
- production infrastructure,
- separate authentication.

11. Task 1 — Why This Is the Beginning of a Harness

A useful mental model is:

Repository
 + project instructions
 + specifications
 + architecture
 + tests
 + evaluation boundaries
 + tools
 + verification
 + feedback
 = engineering environment around the agent

However, we should not claim that `CLAUDE.md` alone is a complete harness.

A stronger harness develops incrementally through:

- explicit specifications,
- deterministic validation,
- permissions,
- hooks where appropriate,
- tests,

- evaluation suites,
- repository state,
- feedback mechanisms,
- and controlled tool access.

This is why the project should be described as moving from a **project engineering foundation** toward a stronger **harness engineering environment**.

12. Task 2 — Formalise the UISpec Model Contract

Task 2 — Complete Prompt

Proceed with the proposed next smallest task.

Formalise the UISpec model contract as a machine-checkable JSON Schema.

Create:

`specs/schemas/ui-spec.schema.json`

Use JSON Schema Draft 2020-12.

The schema must represent the contract currently defined in:

`specs/model-contracts.md`

Do not redesign the contract or introduce new capabilities.

Before writing the schema:

1. Read:

- `CLAUDE.md`
- `docs/business-problem.md`

- docs/architecture.md
- specs/spec.md
- specs/model-contracts.md

2. Identify any ambiguity or contradiction between the prose contract and what can be represented in JSON Schema.

3. If an ambiguity affects the schema design, document it clearly rather than silently inventing a rule.

The schema must cover:

- UISpec top-level structure
- Required fields
- Allowed component types
- Component-specific properties
- String length constraints
- Enum constraints
- Required properties
- Additional-property restrictions
- FeatureGrid child rules
- Hero child/container rules if defined by the contract
- InfoPanel child/container rules if defined by the contract
- CTA action enum/table
- Node-count limits where JSON Schema can express them
- Array size limits where applicable

Use JSON Schema capabilities such as:

- \$schema
- \$defs

- \$ref
- enum
- const
- required
- additionalProperties
- minLength
- maxLength
- minItems
- maxItems
- oneOf / anyOf where appropriate

Do NOT add application code yet.

Do NOT add runtime dependencies.

Do NOT build the React application.

Do NOT build the FastAPI application.

Do NOT integrate a model.

Do NOT create authentication.

Do NOT create document processing or risk functionality.

After creating the schema:

1. Validate that the schema itself is syntactically valid.
2. Check that every v0 component in the model contract is represented.
3. Check that every allowed CTA action is represented.

4. Check that the schema does not accidentally permit arbitrary component types.
5. Report any contract ambiguity discovered.
6. Summarize exactly what was created.

Stop after this task. Do not proceed to the validator or tests until explicitly instructed.

After Task 1, the next smallest implementation task was:

Convert the written UISpec model contract into a machine-checkable JSON Schema.

Created:

specs/schemas/ui-spec.schema.json

The schema uses JSON Schema Draft 2020-12.

It represents:

- UISpec top-level structure,
- required fields,
- approved component types,
- component-specific properties,
- string limits,
- enum constraints,
- required properties,
- additional-property restrictions,
- container child rules,
- CTA action rules,
- array limits,
- structural node-related limits that can be represented by JSON Schema.

The schema also uses:

- ``$defs``
- ``$ref``
- ``oneOf``
- ``const``
- ``enum``
- ``required``
- ``additionalProperties``
- ``minLength``
- ``maxLength``
- ``minItems``
- ``maxItems``

13. Task 2 — What JSON Schema Does and Does Not Do

The important distinction is:

```
Written Model Contract
  ↓
Machine-Checkable JSON Schema
  ↓
Deterministic structural validation
```

JSON Schema can answer questions such as:

- Is the component name allowed?
- Is a required property missing?
- Is a property the correct type?
- Is a CTA action one of the allowed actions?
- Is an array too large?
- Is an unexpected property present?

But some rules require application logic.

The Task 2 work deliberately delegated these to the future validator:

1. Total component node count ≤ 30 .
2. Serialized UISpec size ≤ 8 KB.
3. ``ProgressIndicator.current < len(steps)``.
4. ``meta.generated_for`` must equal the corresponding input context values.

This is an important engineering lesson:

A schema is a powerful structural contract, but it is not automatically the entire runtime policy engine.

14. Task 2 — Contract Ambiguities and Explicit Decisions

The schema work surfaced several ambiguities rather than silently inventing rules.

Hero children

The prose contract allowed a small number of children and described the intended child types. The schema resolved this strictly to the contract-defined types.

``meta.generated_for``

The prose implied a context echo. The machine-checkable schema tightened the field to the defined context enums, while exact equality with the input remained a validator responsibility.

Empty strings

The schema uses `minLength: 1` where strings must not be empty.

Progress indicator

The schema can bound `current`, but it cannot conveniently express the exact relationship:

```
current < len(steps)
```

That relationship remains a runtime validation rule.

URL-bearing structures

The v0 contract contains no active URL-bearing component props, so URL host allow-list enforcement remains dormant rather than invented.

15. Task 2 — The Important Mental Model

Task 2 does **not** make the AI deterministic.

It makes the **contract checkable**.

AI remains probabilistic

↓

AI produces structured output

↓

Schema checks structural validity

This is the beginning of a deterministic envelope around probabilistic behaviour.

16. Task 2 Checkpoint

Before beginning Task 3, the project was deliberately checkpointed.

The verified history became:

```
3824b07 Task 2: Formalise UI Spec model contract
e289754 Task 1: Establish project foundation
2961344 Initial commit
```

Both Task 1 and Task 2 were pushed to `origin/main`.

Why checkpoint before the next task?

A checkpoint gives the team a known-good recovery point.

If a later implementation introduces a problem, the repository can be returned to a known state.

The checkpoint process also teaches an important agentic engineering lesson:

When repository reality does not match the assumptions in a prompt, stop and inspect before making state-changing decisions.

In the Task 2 checkpoint, the expected history did not initially exist exactly as assumed. Instead of silently creating a misleading history, the discrepancy was surfaced and resolved through explicit commits.

17. Task 3 — Deterministic UISpec Validator

Exact Claude Code Prompt Used for Task 3

The following is the complete Task 3 implementation prompt recorded in the Task 3 guide:

We are now starting Task 3 of the Customer Onboarding AI project.

The repository is currently at a verified and remote-synchronized Task 2 checkpoint.

IMPORTANT:

Do NOT start Task 4.

This task is ONLY about implementing the deterministic UISpec validation layer and its tests.

Do not build the React application.

Do not build the FastAPI application.

Do not integrate an AI model.

Do not create a model gateway.

Do not create authentication.

Do not create document processing.

Do not create customer registration.

Do not create risk AI.

Do not create approval workflows.

Do not create production infrastructure.

Do not redesign the existing model contract.

Do not add capabilities that are not required by the current specification.

TASK 3 OBJECTIVE

Build a deterministic validator for the UISpec model contract.

The validator will eventually sit between the probabilistic AI model output and the controlled UI renderer.

The conceptual boundary is:

Raw model output

↓

UISpec validation

↓

Approved UISpec

↓

Controlled renderer

The validator must make a deterministic PASS/REJECT decision.

1. READ THE EXISTING PROJECT FIRST

Before creating or modifying anything, read:

- CLAUDE.md
- README.md
- docs/business-problem.md
- docs/architecture.md
- specs/spec.md
- specs/model-contracts.md
- specs/schemas/ui-spec.schema.json
- specs/schemas/README.md
- tests/README.md
- evals/README.md

Do not assume the contents.

Use the existing documents as the source of truth.

2. INSPECT THE CURRENT GIT STATE

Confirm:

- current branch
- current commit
- working tree status
- current repository structure

The repository should start from the Task 2 checkpoint:

3824b07 Task 2: Formalise UISpec model contract

Do not modify anything until you have inspected the current state.

3. DEFINE THE VALIDATOR RESPONSIBILITY

The validator must be deterministic.

For the same input, it must always produce the same validation result.

The validator must NOT:

- call an AI model
- make network calls
- access a database
- execute arbitrary code
- modify the UISpec
- "repair" invalid model output
- invent missing fields
- silently change values
- make business decisions

- approve customers
- render UI

The validator should answer:

"Does this UISpec comply with the defined contract?"

4. JSON SCHEMA VALIDATION

Use:

specs/schemas/ui-spec.schema.json

as the structural source of truth.

The validator should use an appropriate JSON Schema validation library.

Do not rewrite the schema into application code.

The schema should remain the authoritative structural contract.

The validator should report useful validation failures rather than only returning a generic failure message.

5. IMPLEMENT THE RULES DELEGATED FROM TASK 2

The validator must implement the constraints that JSON Schema could not fully express.

These were explicitly documented in:

specs/schemas/README.md

At minimum, implement:

A. TOTAL NODE COUNT

The total number of UISpec nodes across the component tree must not exceed 30.

B. SERIALIZED UISPEC SIZE

The serialized UISpec must not exceed 8 KB.

C. PROGRESS INDICATOR RELATIONSHIP

For ProgressIndicator:

current must satisfy:

current < len(steps)

The validator must reject invalid values.

D. GENERATED CONTEXT EQUALITY

The values in:

`meta.generated_for`

must match the corresponding values from the input `ContextPayload`.

Do not merely check that the values are valid enums.
They must match the supplied context.

6. URL HOST ALLOW-LIST

Review the existing contract and schema documentation carefully.

If the current `v0` contract contains no URL-bearing component properties, do NOT invent URL validation functionality.

Document that the URL host allow-list remains dormant/not applicable for the current `v0` component vocabulary.

Do not introduce URL fields merely to implement this rule.

7. VALIDATION RESULT

Define a clear deterministic validation result.

For example, conceptually:

VALID

or

INVALID

with structured error information.

The exact shape should be derived from the existing project conventions rather than invented unnecessarily.

A useful validation failure should identify:

- what failed
- where it failed
- why it failed

For example:

`component[2].props.headline`

or:

`component[1].children[3]`

Avoid leaking implementation-specific stack traces as the primary validation result.

8. DO NOT AUTO-REPAIR

The validator must reject invalid output.

It must NOT:

- truncate strings
- remove unknown properties
- replace invalid component types
- change CTA actions
- fix `ProgressIndicator.current`
- remove excess nodes
- alter `generated_for`
- silently make the model output valid

The principle is:

INVALID → REJECT

not:

INVALID → REPAIR → ACCEPT

9. TESTS

Create deterministic automated tests for the validator.

The tests must cover at least:

VALID CASES

1. Valid minimal `UISpec`
2. Valid Hero + CTA + Text
3. Valid `FeatureGrid` with valid `FeatureCard` children
4. Valid refusal Alert
5. Valid `ProgressIndicator`

INVALID CASES

6. Unknown component type
7. Unknown component property
8. Invalid CTA action
9. Invalid component structure
10. More than 30 total nodes
11. `UISpec` larger than 8 KB
12. `ProgressIndicator.current` $\geq$ `len(steps)`
13. `meta.generated_for` does not match `ContextPayload`
14. Invalid schema structure
15. Attempted unsupported URL-bearing structure if applicable to the contract

For every invalid test, verify that the validator rejects the input.

For every valid test, verify that the validator accepts the input.

10. TEST DETERMINISM

Include at least one test demonstrating that the same input produces the same validation result when validated repeatedly.

The validator must not contain randomness or model-dependent behavior.

11. KEEP TESTS CLOSE TO THE CONTRACT

Do not create a large testing framework.

Keep the test suite small and understandable.

Tests should clearly demonstrate:

Contract rule
↓
Input
↓
Expected result

Prefer readable fixtures/examples over unnecessary abstraction.

12. DEPENDENCY DISCIPLINE

Before adding a dependency:

1. Check whether an existing dependency already supports JSON Schema validation.
2. If no suitable dependency exists, choose one small, well-established JSON Schema validation library appropriate for the implementation language.
3. Do not add unrelated dependencies.

Do not introduce React, FastAPI, model SDKs, database libraries, or infrastructure dependencies.

13. DOCUMENTATION

Update the relevant test/validator documentation only as necessary to explain:

- what the validator does
- what JSON Schema handles
- what runtime validation handles
- what the validator intentionally does NOT do

Do not rewrite the existing model contract.

14. VERIFY THE IMPLEMENTATION

After implementation:

Run the complete validator test suite.

Also verify:

- all existing Task 2 schema checks still pass
- valid examples pass
- invalid examples fail
- no test relies on an AI model
- no network access is required
- validator results are deterministic

15. REVIEW THE DIFF

Before finishing:

Run:

```
git status
git diff --stat
git diff
```

Review all changes.

Confirm that the implementation is limited to Task 3.

Do not modify unrelated files.

16. IMPORTANT STOPPING RULE

Do NOT create a Git commit yet.

Do NOT push anything.

At the end of Task 3 implementation, stop and report:

1. What was implemented
2. Files created
3. Files modified
4. Dependencies added, if any
5. Validation rules implemented
6. Tests created
7. Test results
8. Existing Task 2 checks status
9. Any ambiguities discovered
10. Current git status
11. Proposed next checkpoint

Do not start Task 4.

Do not commit the changes.

We will review the Task 3 implementation before creating the next checkpoint.

Task 3 added the deterministic gate between model output and future rendering.

The actual implementation is:

```
specs/schemas/ui-spec.schema.json
      ↓
domain/validator.py
      ↓
VALID / INVALID
```

The validator is intentionally:

- deterministic,
- local,
- side-effect free,
- independent of an AI model,
- independent of network calls,
- independent of databases,
- independent of React.

18. Task 3 — Two Validation Layers

Every UISpec must pass both layers.

Layer 1 — JSON Schema

Structural rules include:

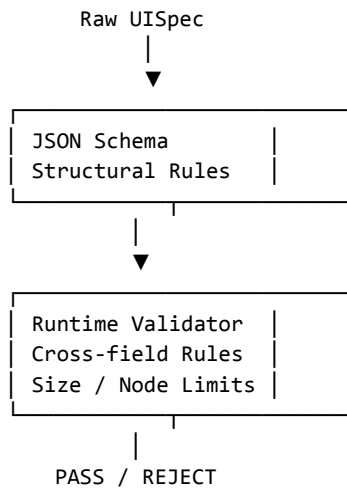
- correct data types,
- required fields,
- approved components,
- approved properties,
- approved CTA actions,
- string length limits,
- array limits,
- container-child rules.

Layer 2 — Runtime Validator

Task 3 adds:

- total component node count ≤ 30 ,
- serialized UISpec ≤ 8 KB,
- ``ProgressIndicator.current < len(steps)``,
- ``meta.generated_for`` equals the supplied ``ContextPayload``.

Conceptually:



A spec that passes the schema but fails a runtime rule is still rejected.

19. What Does “Deterministic” Mean?

Deterministic means:

Given the same input and the same validator version, the result is reproducible.

For example:

Input A → VALID
Input A → VALID
Input A → VALID

and:

Input B → INVALID
Input B → INVALID
Input B → INVALID

The validator itself does not call an AI model and does not introduce randomness.

This does ****not**** mean the upstream AI becomes deterministic.

The distinction is:

Probabilistic model
↓
Variable proposal
↓
Deterministic validator
↓
Predictable acceptance/rejection boundary

20. What Is a Node?

For the v0 validator:

A node is one component instance in the UISpec component tree.

For example:

```
Hero
├── Heading
├── Text
└── CTA
```

contains four component nodes.

The following do ****not**** count as nodes:

- the outer UISpec envelope,
- JSON properties,
- metadata,
- strings,
- arrays themselves.

Exactly 30 component instances are allowed.

31 or more are rejected.

The current v0 schema is actually much more restrictive than the 30-node defensive limit because the currently allowed containers have bounded children and those children are mostly leaves.

The 30-node rule therefore acts as a future-proof backstop.

21. The 8 KB Limit

The serialized UISpec must not exceed 8 KB.

This is different from the node limit.

Rule • What it controls

≤30 nodes • Component-tree complexity

≤8 KB • Serialized response size

A UISpec could contain relatively few components but still carry very large strings.

Therefore both dimensions are bounded independently.

22. Context Equality

The model receives a `ContextPayload`.

The model must echo the relevant context in:

```
meta.generated_for
```

The validator checks that the returned values match the supplied input.

For example:

```
ContextPayload
segment = enterprise
product_interest = banking
region = APAC
```

must not return:

```
meta.generated_for
segment = enterprise
product_interest = banking
region = Europe
```

The validator rejects the mismatch.

This creates an explicit consistency check between:

what the application asked the model to generate

and:

what the model says it generated

23. ProgressIndicator Runtime Rule

Suppose:

```
steps = [
    "Customer Setup",
    "Document Check",
    "Risk Review"
]
```

There are three steps.

Because indexing starts at zero:

```
current = 0 → valid
current = 1 → valid
current = 2 → valid
current = 3 → invalid
```

The runtime rule is:

```
current < len(steps)
```

This is a good example of a rule that is conceptually simple but better handled by runtime validation than by the JSON Schema alone.

24. Reject Versus Repair

Task 3 deliberately chose ****rejection rather than automatic repair****.

The validator does not:

- truncate long strings,
- remove unknown properties,
- change invalid actions,
- clamp invalid progress values,
- drop excess nodes,
- rewrite mismatched context.

Instead:

```
INVALID MODEL OUTPUT
  ↓
  REJECT
  ↓
LOG / MEASURE
  ↓
Safe fallback or error handling
```

The distinction matters because silent repair hides model behaviour.

If invalid outputs are silently fixed, it becomes harder to answer:

“How often does the model actually produce invalid output?”

Keeping the original failure visible makes model quality measurable.

25. Structured Validation Results

The validator does not return only:

false

It returns structured errors.

Conceptually:

```
INVALID
├── path
├── rule
└── message
```

Example:

```
path: root.props.action
rule: enum
message: 'delete_account' is not one of
['start_onboarding', 'learn_more', 'contact_sales']
```

This is useful for:

- developers,
- logs,
- automated tests,
- future evaluation systems,
- operational diagnostics.

26. Task 3 — Tests

Task 3 created:

tests/domain/test_validator.py

The test suite covers valid and invalid UISpecs.

Valid cases include:

- minimal UISpec,
- Hero + CTA + Text,
- FeatureGrid,
- refusal Alert,
- ProgressIndicator.

Invalid cases include:

- unknown component,
- unknown property,
- invalid CTA action,
- invalid component structure,
- more than 30 nodes,

- more than 8 KB,
- invalid progress index,
- context mismatch,
- invalid top-level structure,
- unsupported URL-bearing structure where applicable.

Additional checks cover:

- repeated deterministic validation,
- no mutation,
- no automatic repair,
- structured validation-result shape.

The Task 3 suite completed with:

24 passed

Task 2's schema verification was also re-run successfully:

12 / 12 checks passed

27. Task 3 — Actual Files

Task 3 added the first Python implementation:

```
domain/
├── __init__.py
└── validator.py

tests/
├── domain/
│   └── test_validator.py
└── pytest.ini
requirements.txt
```

The main dependency roles are:

- `jsonschema` — machine-checkable contract validation.
- `pytest` — deterministic automated testing.

The validator does not replace the schema.

The relationship is:

```
Model Contract
  ↓
JSON Schema
  ↓
Python Validator
```

↓
Validation Result

28. Architecture Review Before Task 4

After Task 3, implementation paused for an architecture review.

This pause is intentional.

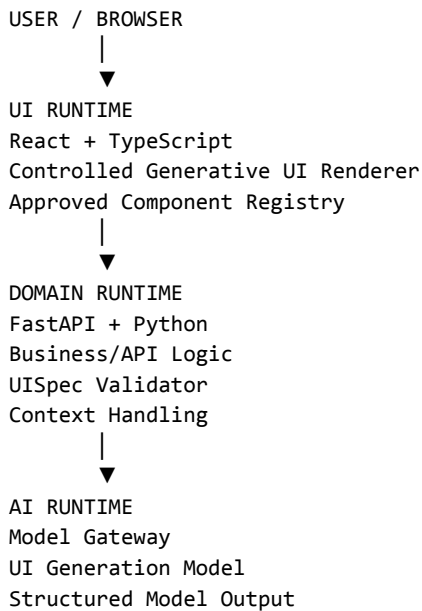
AI-native engineering should not become:

Task finished
↓
Immediately write more code
↓
Repeat

Instead:

Small implementation
↓
Verification
↓
Architecture review
↓
Checkpoint
↓
Next smallest implementation

The review confirmed the current runtime architecture:

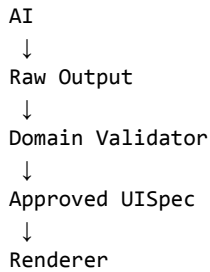


29. Architecture Review — The Critical Boundary

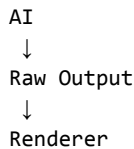
The architecture review established a strict rule:

*****The renderer receives only an already-approved UISpec.*****

Therefore:



not:

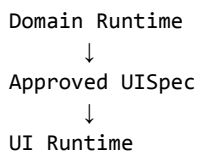


At the architecture-review stage, this was a logical runtime boundary.

The Python validator and React renderer were ****not yet physically connected through a production API****.

That distinction is important.

The architecture describes the eventual flow:



but the actual runtime/API integration comes later.

30. Architecture Review — Contract Vocabulary vs Executable Registry

Another important distinction was clarified.

The model contract defines the approved ****vocabulary****:

Hero
Heading
Text

FeatureCard
FeatureGrid
CTA
Alert
InfoPanel
ProgressIndicator

The JSON Schema makes that vocabulary structurally checkable.

The React registry is a separate executable artifact:

`ui/src/registry.ts`

It maps approved names to actual React implementations.

Therefore:

```
MODEL CONTRACT
  ↓
JSON SCHEMA
  ↓
VALIDATOR
  ↓
APPROVED UISPEC
  ↓
RUNTIME COMPONENT REGISTRY
  ↓
REACT RENDERER
```

The registry should not become a second, independently invented model contract.

31. Task 4 — Controlled UISpec Renderer

Task 4 Claude Code Implementation Prompt

****Source note:**** The retrieved project record preserves the Task 4 implementation requirements and stopping rules, but does not preserve the original assistant message word-for-word. The block below records the Task 4 implementation brief used for the completed renderer work without presenting reconstructed wording as a verbatim transcript.

Task 4 – Controlled UISpec Renderer

We are now proceeding with the next smallest implementation increment after the Task 3 deterministic validator and the architecture review.

Before implementation:

1. Read the existing project instructions and specifications:
 - CLAUDE.md
 - README.md
 - docs/business-problem.md

- docs/architecture.md
- specs/spec.md
- specs/model-contracts.md
- specs/schemas/ui-spec.schema.json
- specs/schemas/README.md
- domain/validator.py
- relevant Task 3 tests and documentation

2. Inspect the current repository state and confirm the Task 3 implementation is understood.

Task 4 objective:

Build the controlled UISpec renderer for the UI Runtime.

The renderer must accept only an already-approved UISpec.

The conceptual contract is:

render(approvedUISpec)

NOT:

render(rawAIOutput)

Create the React + TypeScript renderer and an explicit approved component registry for the v0 UISpec vocabulary:

- Hero
- Heading
- Text
- FeatureCard
- FeatureGrid
- CTA
- Alert
- InfoPanel
- ProgressIndicator

The component registry must be explicit and controlled.

Do not use:

- dynamic imports based on model-supplied component names
- eval
- Function constructors
- arbitrary executable code
- arbitrary HTML execution
- model-generated React/HTML/JavaScript
- arbitrary module lookup

The renderer should:

- resolve components only through the approved registry
- reject unsupported component names
- recursively render approved child structures
- use developer-authored React components
- avoid mutating the input UISpec
- keep component props controlled
- preserve the distinction between the model contract and the executable component registry

Use the existing JSON Schema/model contract as the contract source.
Do NOT create a second independent JSON Schema in TypeScript.
Do NOT redesign the UISpec contract.

Create deterministic tests covering:

- registry integrity
- valid rendering
- unsupported component rejection
- deterministic behaviour
- non-mutation
- HTML escaping / safe text rendering
- protection against unknown props reaching the DOM
- absence of dangerous execution mechanisms
- absence of dynamic component loading from model input

Do not build:

- FastAPI
- runtime model integration
- model gateway
- authentication
- document processing
- customer registration
- risk AI
- human approval workflow
- production infrastructure
- physical Python-validator-to-React API integration

Important architectural boundary:

AI model



Raw UISpec



Domain validator



Approved UISpec



Controlled React renderer



Browser

The Task 3 Python validator and Task 4 React renderer are not yet physically connected through a runtime API. Implement the renderer boundary only.

Before finishing:

- run the renderer test suite
- rerun the Task 3 Python tests
- rerun the Task 2 schema verification checks
- review git status and diff
- confirm no unrelated functionality was added

Do NOT commit or push yet.

Stop after Task 4 and report:

1. What was implemented
2. Files created

3. Dependencies added
4. Registry design
5. Renderer design
6. Security controls
7. Test results
8. Regression results
9. Current git status
10. Any observations or ambiguities
11. Proposed checkpoint

Do not proceed to the next task automatically.

Task 4 implemented the next boundary: rendering approved UISpecs through known React components.

The task did **not** add:

- FastAPI,
- runtime model integration,
- authentication,
- document processing,
- risk AI,
- approval workflow,
- production infrastructure.

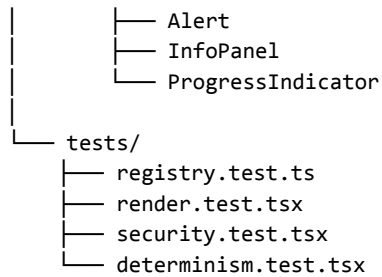
The focus was only:

Given an approved UISpec, render it through an explicit, controlled component registry.

32. Task 4 — React / TypeScript Foundation

Task 4 created:

```
ui/
├── package.json
├── tsconfig.json
├── vitest.config.ts
├── README.md
├──
├── src/
│   ├── types.ts
│   ├── errors.ts
│   ├── registry.ts
│   ├── render.tsx
│   └── components/
│       ├── Hero
│       ├── Heading
│       ├── Text
│       ├── FeatureCard
│       ├── FeatureGrid
│       └── CTA
```



The TypeScript layer represents the same v0 UISpec vocabulary in executable application code.

The important point is not duplication of the schema.

The TypeScript types are the renderer's programming-language representation of an already-defined contract.

33. Task 4 — The Approved Component Registry

The registry is explicit and frozen.

Conceptually:

```

{
  Hero,
  Heading,
  Text,
  FeatureCard,
  FeatureGrid,
  CTA,
  Alert,
  InfoPanel,
  ProgressIndicator
}

```

The registry contains exactly the approved nine component names.

There is no:

dynamic import

no:

eval(...)

no:

Function(...)

and no arbitrary module lookup based on model-supplied strings.

The renderer therefore cannot simply decide:

“The model asked for a component named X, so I will dynamically load X.”

Instead:

```
model component name
    ↓
explicit registry lookup
    ↓
known React component
```

34. Task 4 — Renderer Contract

The renderer exposes the conceptual contract:

```
render(approvedUISpec)
```

It:

1. receives the UISpec,
2. looks up the component in the approved registry,
3. rejects unknown component names,
4. recursively renders approved children,
5. uses React components written by developers.

It does not:

- execute model-generated code,
- dynamically import arbitrary modules,
- interpret JavaScript,
- inject arbitrary HTML,
- modify the input UISpec.

The renderer assumes the domain layer has already performed full validation, while still defending its own registry boundary.

35. Task 4 — Security-Oriented Tests

Task 4 created 19 tests.

They cover:

- registry integrity,
- happy-path rendering,
- unsupported component rejection,
- deterministic rejection,
- HTML escaping,
- prevention of unknown props reaching the DOM,
- absence of `dangerouslySetInnerHTML`,
- absence of `eval`,
- absence of `Function`,
- absence of dynamic imports,
- determinism,
- non-mutation.

The Task 3 Python tests were also re-run successfully:

24 passed

Task 2 schema verification remained:

12 / 12 passed

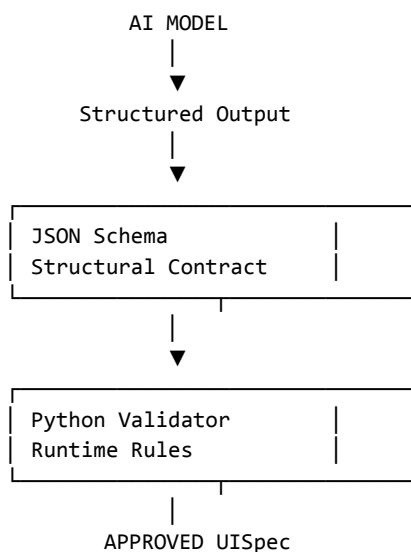
Task 4's renderer tests:

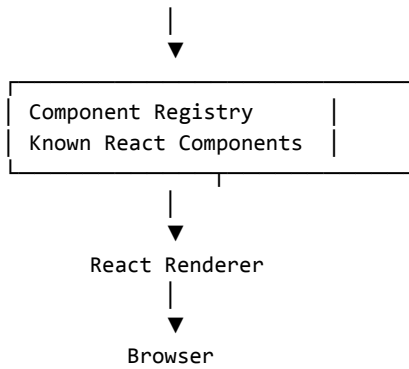
19 passed

The overall project therefore had deterministic evidence across the layers already implemented.

36. The Complete Control Chain After Task 4

The architecture now has a much clearer control chain:





The model does not cross directly into the executable UI layer.

37. JSON Schema and Component Registry Are Not the Same Thing

This distinction is worth making explicit because both contain component names.

JSON Schema

Answers:

“Is this UISpec structurally allowed by the model contract?”

Component Registry

Answers:

“Which actual React implementation corresponds to this approved component name?”

They are related:

```
Schema vocabulary
  ↓
Validator
  ↓
Registry vocabulary
```

but they have different responsibilities.

The schema is the structural contract.

The registry is the executable rendering boundary.

38. Deterministic Tests vs AI Evals

The project deliberately keeps these separate.

Deterministic Tests • AI Evals

Check software correctness • Measure model behaviour

Repeatable • Statistical / sample-based

No model required • Model usually involved

Example: unknown component is rejected • Example: model chooses an appropriate component

Lives in `tests/` • Lives in `evals/`

An AI response can be structurally valid and still be poor.

For example:

Valid JSON
+
Approved components
+
Valid props

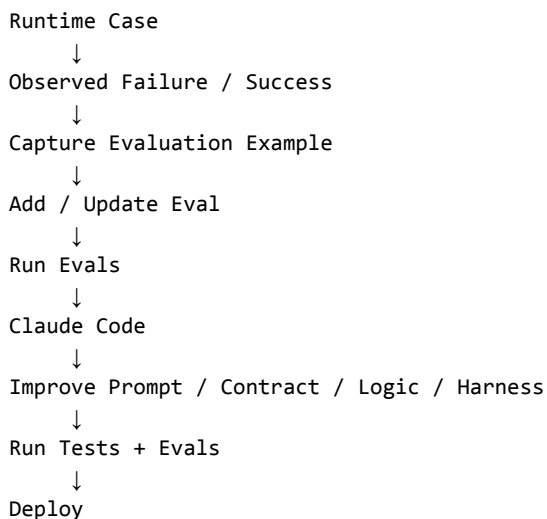
does not automatically mean:

Useful landing page
Relevant wording
Good contextual adaptation
Truthful claims
Appropriate UX

Those quality questions belong to AI evaluation.

39. Feedback as a Future Engineering Loop

The architecture eventually becomes:



The important idea is that production experience becomes engineering evidence.

A model failure should not simply disappear after a one-off fix.

It can become:

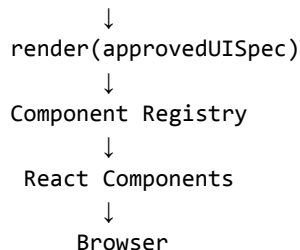
- a regression test where deterministic,
- an evaluation case where probabilistic,
- a contract clarification,
- a prompt/context improvement,
- or a harness improvement.

Task 5 — Minimal Runnable React UI

Task 5 takes the controlled renderer completed in Task 4 and proves that an approved, developer-authored UISpec can produce a real browser-runable Customer Onboarding UI. The purpose of this increment is deliberately narrow: prove the renderer-to-browser path without introducing runtime AI, FastAPI, authentication, document processing, risk scoring, approval workflows, or production infrastructure.

5.1 The Task 5 Runtime Path

Developer-authored approved UISpec



The key architectural rule remains unchanged: the application renders an approved UISpec, not raw model output. Task 5 therefore demonstrates the controlled UI path using fixtures that are authored by the developer and treated as already-approved UISpec representations.

5.2 What Was Implemented

- Updated `.gitignore` to exclude generated browser build output.
- Updated `ui/package.json` and `ui/package-lock.json` with the minimum Vite/build tooling required to run the browser application.
- `ui/tests/app.test.tsx` — tests confirming the application uses the controlled renderer path.
- `ui/tests/fixture.test.tsx` — deterministic tests for the fixtures.
- `ui/src/fixtures/customer-onboarding.ts` — developer-authored, typed UISpec fixtures.
- `ui/src/styles.css` — intentionally small styling layer for the demonstration UI.

- ``ui/src/App.tsx`` — application shell that maps the approved fixtures through the existing ``render(spec)`` function.
- ``ui/src/main.tsx`` — React application entry point using ``createRoot`` and ``StrictMode``.
- ``ui/vite.config.ts`` — Vite and React configuration.
- ``ui/index.html`` — minimal Vite browser shell.

5.3 The Developer-Authored UISpec Fixtures

Four small UISpec fixtures were used to demonstrate the renderer. They are not model responses. They are developer-authored examples of UISpecs that have already passed the contract assumptions for this demonstration.

- ``infoSpec`` — InfoPanel containing Heading and Text children.
- ``featuresSpec`` — FeatureGrid containing FeatureCard children.
- ``progressSpec`` — ProgressIndicator demonstration.
- ``heroSpec`` — Hero with Heading/Text/CTA-oriented content.

Together these fixtures exercise all nine approved component types except Alert. Alert is primarily used for the refusal/error shape rather than the normal landing-page demonstration.

5.4 Why the Fixtures Are Separate

During Task 5, an architectural constraint in the existing v0 contract became visible. The contract permits one root node for a UISpec and does not yet define a general page-level container that can hold arbitrary sections. Therefore, a single UISpec cannot legitimately combine a Hero, FeatureGrid, and other independent sections under one root.

The implementation therefore renders several valid single-root UISpecs separately. This preserves the existing model contract instead of silently changing it. If a future version requires one UISpec to represent a complete multi-section page, that should be handled as a deliberate contract change rather than an implementation workaround.

5.5 Renderer Integration at the Application Boundary

The application shell uses the existing controlled renderer for every fixture. App.tsx does not import the component registry directly, does not call ``createElement``, and does not manipulate the DOM. The renderer remains the single path from an approved UISpec to React components.

```
{customerOnboardingFixtures.map((spec, i) => (
  <Fragment key={i}>{render(spec)}</Fragment>
))}
```

This is important because the application is demonstrating the architectural boundary rather than merely producing a visually correct page.

5.6 Important Boundary Clarification

The Python UISpec validator from Task 3 is still not physically connected to the React application. Task 5 does not create a Python-to-React API or bypass that separation. The

fixtures used here are developer-authored approved UISpec representations. Physical domain/API integration is intentionally left for the next runtime increment.

Task 3: Python validator

↓ (architectural relationship only at this stage)

Task 4: Controlled React renderer

↓

Task 5: Runnable browser UI

↓

Task 6: Domain/API runtime boundary

5.7 Task 5 Testing and Verification

- Security checks continued to confirm that the application does not introduce `eval()`, `Function()`, dynamic imports, arbitrary component resolution, or arbitrary DOM execution.
- Vite development server started successfully and the browser shell returned HTTP 200 during the smoke test.
- ``npm run build``: passed.
- Task 2 JSON Schema verification: 12/12 checks passed.
- Task 3 validator regression tests: 24/24 passed.
- Task 4 renderer regression tests: 19/19 passed.
- Task 5 UI tests: 38/38 passed.

5.8 What Task 5 Deliberately Does Not Build

- No production infrastructure, deployment, or production observability.
- No human approval workflow.
- No risk scoring.
- No cross-document validation AI.
- No document processing.
- No customer registration workflow.
- No authentication subsystem.
- No FastAPI integration.
- No model gateway.
- No runtime AI/model integration.

Task 5 — Git Checkpoint

After implementation and verification, Task 5 was checkpointed so the project has a known-good, recoverable state before the next architectural increment.

bc96e12 Task 5: Implement minimal runnable React UI

Previous checkpoint:

1cb43a0 Task 4: Implement controlled UISpec renderer

The Task 5 checkpoint was pushed to `origin/main` and the working tree was confirmed clean and synchronized.

Implement
↓
Verify
↓
Review
↓
Commit
↓
Push
↓
Known-good state

2961344 Initial commit

e289754 Task 1: Establish project foundation

3824b07 Task 2: Formalise UISpec model contract

6ac1e46 docs: extend walkthrough

31d4674 Task 3: Implement UISpec validator and tests

d957d11 docs: extend walkthrough with Task 3

1cb43a0 Task 4: Implement controlled UISpec renderer

bc96e12 Task 5: Implement minimal runnable React UI

Current verified Git history:

40. Git Checkpoints and Recoverability

The verified implementation history currently includes:

31d4674 Task 3: Implement UISpec validator and tests
6ac1e46 docs: extend project walkthrough
3824b07 Task 2: Formalise UISpec model contract
e289754 Task 1: Establish project foundation
2961344 Initial commit

Tasks 4, 5, and 6 are now checkpointed. The latest verified state is Task 6.

The important process remains:

Implement
↓
Verify
↓
Review
↓
Checkpoint
↓
Push

A checkpoint is more than a Git housekeeping step.

It is a statement:

“The project has reached a known, verified state here.”

41. What Has Not Been Built Yet

Even after Task 5, the following are intentionally outside the completed scope:

- FastAPI runtime integration,
- physical Python-validator → API → React integration,
- runtime model gateway,
- actual model API call,
- document processing,
- customer registration,
- cross-document validation AI,
- risk AI,
- human approval workflow,
- production authentication,
- production infrastructure,
- complete onboarding workflow,
- production observability and deployment.

This is not unfinished work in the negative sense.

It is deliberate incremental engineering.

Each missing capability becomes a future, separately reviewable task.

42. Current Architecture and Project Status

Area • Status

Business problem • Defined

AI-native engineering concept • Defined

Runtime architecture • Defined

Controlled GenUI scenario • Defined

Project foundation • Complete

Model contract • Complete

JSON Schema • Complete

Task 2 checkpoint • Complete

Deterministic validator • Complete

Validator tests • Complete

Architecture review • Complete

Controlled React registry • Implemented

Controlled React renderer • Implemented

Renderer tests • Implemented

Domain/UI API integration • Implemented

FastAPI runtime • Implemented

Model gateway • Not yet

Runtime model integration • Not yet

AI eval suite • Not yet

Feedback loop • Not yet

Production architecture • Not yet

43. The Incremental Engineering Pattern

The project demonstrates a repeatable pattern:

Define
↓
Specify
↓
Formalise
↓
Validate
↓
Test
↓
Review
↓
Render
↓
Integrate
↓
Evaluate
↓
Learn

↓
Improve

Each increment should be:

- small,
- understandable,
- testable,
- reviewable,
- reversible,
- explicitly scoped.

This is particularly important when an AI coding agent is involved.

A large prompt that asks for the entire system can produce a large change that is difficult to understand.

Small prompts create smaller evidence-bearing steps.

44. The Core AI-Native Engineering Lesson

Traditional software engineering asks:

“Does the code behave correctly?”

AI-native engineering must additionally ask:

“What happens when a probabilistic component proposes something unexpected?”

The answer in this project is not:

“Trust the model more.”

The answer is:

Give the model a constrained contract

↓

Validate its output

↓

Reject invalid proposals

↓

Render only approved structures

↓

Test deterministic boundaries

↓

Evaluate probabilistic behaviour

↓

Feed failures back into engineering

This is the central pattern the Customer Onboarding example is designed to teach.

45. One-Sentence Summary of Each Milestone

Initial discussion

Establish the business problem and explain why specification-driven engineering is needed before asking Claude Code to build software.

Architecture guide

Explain how frontend/backend, UI/Domain/AI runtime views, harness engineering, Claude Code, tests, evals, feedback, and production fit together.

First implementation guide

Turn the abstract AI-native architecture into a small, visible Controlled Generative UI scenario.

Task 1

Create the project engineering foundation.

Task 2

Turn the prose model contract into a machine-checkable JSON Schema.

Task 2 checkpoint

Preserve the verified state before introducing runtime validation.

Task 3

Create the deterministic validation boundary around probabilistic model output.

Architecture review

Confirm that approved UISpecs, not raw model output, are the only input allowed into the renderer.

Task 4

Create the explicit React component registry and controlled renderer.

46. The Mental Model to Remember

If the entire project feels complex, remember these four questions:

1. **What does the user see?**

UI Runtime / React.

2. **What controls the business process?**

Domain Runtime / FastAPI and Python.

3. ****What responsibilities involve probabilistic intelligence?****

AI Runtime / model gateway and models.

4. ****What helps the engineering agent build and improve the system safely?****

Harness / specifications / instructions / validation / tests / evals / feedback.

And remember the most important boundary:

```
AI proposes
↓
Contract constrains
↓
Schema validates structure
↓
Validator checks runtime rules
↓
Registry controls executable components
↓
Renderer produces UI
```

47. What Comes Next

The next implementation should be selected only after reviewing the current Task 4 state.

The likely progression is:

```
Task 4
Controlled Renderer
↓
Task 5
Small React Application Shell / Landing Page
↓
Task 6
Domain Runtime / FastAPI Boundary
↓
Task 7
Model Gateway
↓
Task 8
Runtime Model Integration
↓
Task 9
AI Evaluation Suite
↓
Task 10
Feedback and Continuous Improvement
↓
Future
Production Architecture
```

The exact numbering should be decided from the verified repository state rather than assumed in advance.

The project should continue using the same discipline:

*****One small task → verify → review → checkpoint → next task.*****

Current Verified Position After Task 5

The project has now progressed through the first real browser milestone. The controlled UISpec renderer is no longer only a tested library boundary; Task 5 demonstrates that developer-authored approved UISpecs can flow through the registry and renderer into a runnable React application in the browser.

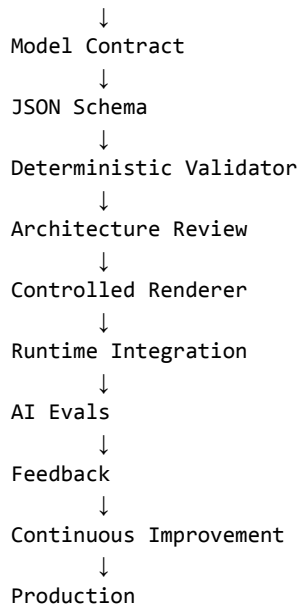
```
Business Problem
↓
Specification
↓
AI-Native Architecture
↓
Model Contract
↓
JSON Schema
↓
Deterministic Validator
↓
Controlled Renderer
↓
Minimal Runnable React UI
↓
Domain Runtime / FastAPI Boundary
```

48. Final Takeaway

The Customer Onboarding AI project is being built as a teaching example of *****AI-native engineering*****, not simply as a demo application.

The sequence matters:

```
Business Problem
↓
Specification
↓
AI-Native Architecture
↓
Controlled Generative UI
↓
Project Foundation
```



The key principle is:

*****Do not make the AI responsible for what deterministic software can safely control.*****

The model can propose intent.

The contract defines the allowed shape.

The schema checks structure.

The validator checks runtime rules.

The component registry controls executable UI capabilities.

The renderer turns approved intent into actual React UI.

Tests verify deterministic behaviour.

Evals measure probabilistic behaviour.

Feedback turns real-world experience into better engineering.

That is the foundation for the larger Customer Onboarding AI system.

Task 6 — Domain Runtime / FastAPI Boundary

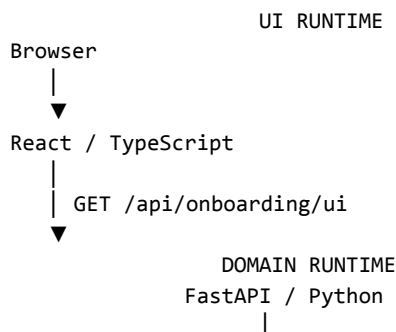
Task 6 established the first real runtime/API boundary between the browser UI and the Python domain runtime. The goal was deliberately narrow: move the approved, developer-authored UISpec behind a FastAPI endpoint, reuse the existing deterministic validator, and have React obtain the UISpec through HTTP.

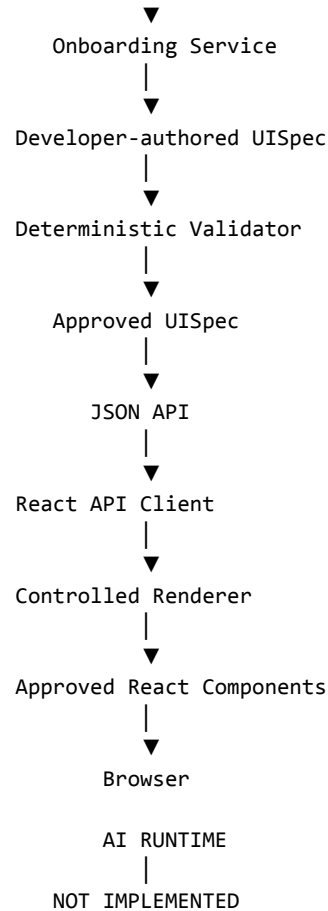
6.1 Where Task 6 Fits



A critical architectural distinction is preserved: FastAPI is the Domain Runtime, not the AI Runtime. The model gateway and runtime AI integration remain intentionally unimplemented.

6.2 The Task 6 Runtime Architecture





6.3 FastAPI Application Structure

```

api/
├── main.py
├── errors.py
├── routes/
│   ├── health.py
│   └── onboarding.py
├── services/
│   └── onboarding.py
└── fixtures/
    └── onboarding_ui.py
  
```

The route layer handles HTTP concerns. The onboarding service contains the small domain operation. The fixture is kept separately so that the future source of the UISpec can change without changing the basic API boundary.

6.4 The Two API Endpoints

```

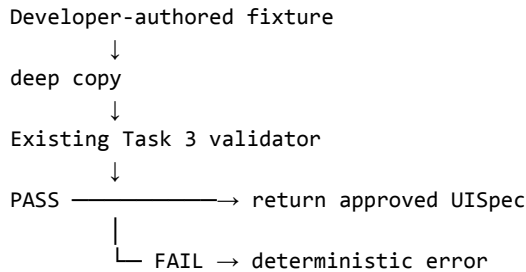
GET /health
→ 200
→ {"status":"ok"}
  
```

```

GET /api/onboarding/ui
→ 200
→ UISpec JSON
  
```

The onboarding endpoint intentionally returns the UISpec itself rather than introducing an unnecessary data/ui envelope. This keeps the runtime seam small and easy to understand.

6.5 The Domain-Service Validation Boundary

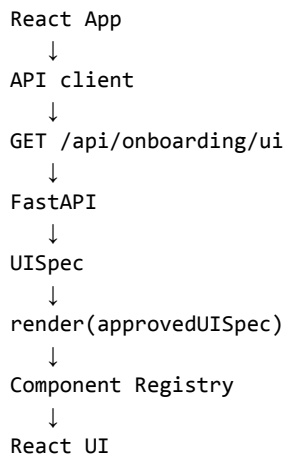


The existing Python validator is reused rather than duplicated. The service validates the fixture before the API returns it. It does not repair invalid data, silently substitute another fixture, or call an AI model to fix the response.

6.6 Why Deep Copy Is Used

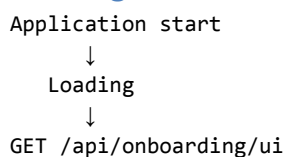
The service deep-copies the developer-authored fixture before validation and return. This keeps the shared fixture object protected from accidental mutation during a request. It is a small defensive measure that helps preserve deterministic behavior.

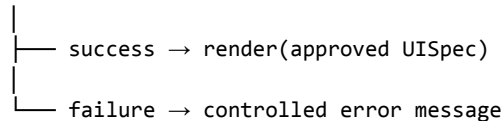
6.7 React API Integration



The existing controlled renderer remains the only rendering path. App.tsx does not import the component registry, create React elements directly, dynamically import components, or interpret API data as executable code.

6.8 Loading, Success, and Error States





A failed API request does not silently fall back to the old Task 5 fixture. The failure remains visible. This makes runtime dependency failures observable instead of hiding them.

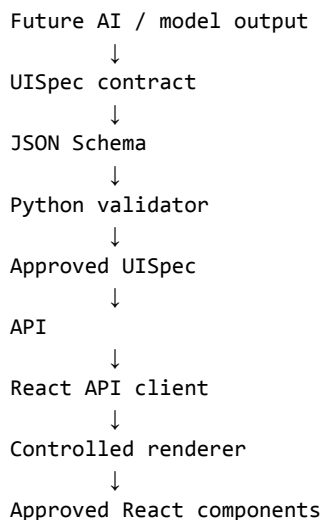
6.9 Local Development Boundary

The local development environment uses React/Vite on port 5173 and FastAPI on port 8000. The Vite development proxy forwards `/api/*` requests to FastAPI. FastAPI also uses an explicit local-development CORS allow-list rather than a wildcard origin.

6.10 Task 6 Testing and Verification

- Python test suite: 34/34 passed — 24 Task 3 validator tests + 10 Task 6 API tests.
- React test suite: 52/52 passed across 7 files.
- Task 2 JSON Schema verification: 12/12 checks passed.
- Production build: TypeScript compilation and Vite build passed.
- GET `/health`: HTTP 200 with `{"status":"ok"}`.
- GET `/api/onboarding/ui`: HTTP 200 with the developer-authored UISpec.
- Live API response fed back through the existing validator: `valid=True`.
- Vite proxy request to `/api/onboarding/ui`: HTTP 200.
- Security regression: Task 4 controls remained intact.

6.11 Security and Control Boundary



Task 6 does not contain a model, but it preserves the boundary that a future model will use. The API is not an arbitrary component execution mechanism.

- No `dangerouslySetInnerHTML`.
- No `eval()`.
- No `new Function()`.
- No `dynamic import()`.

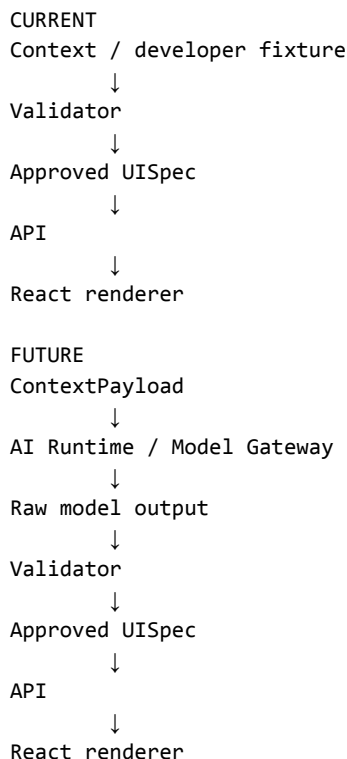
- No arbitrary component resolution.
- No inline JavaScript handlers or javascript: URLs generated by the controlled UI.
- No AI/model SDK imported by the running service.

6.12 What Task 6 Deliberately Does Not Build

- ContextPayload request parameters / Task 6A.
- Model gateway.
- AiClient interface.
- Runtime AI/model integration.
- Authentication.
- Database or persistence.
- Document processing.
- Risk scoring.
- Human approval workflow.
- Production infrastructure.

These omissions are deliberate scope boundaries, not accidental gaps. Each future capability should be introduced as a separately reviewable increment.

6.13 Architectural Observation: The AI Slot Is Now Clearer



Task 6 therefore does not make the application AI-powered. Instead, it creates a clean Domain Runtime boundary into which the future AI Runtime can be introduced without changing the controlled rendering principle.

6.14 Task 6 Git Checkpoint

a55c522 Task 6: Implement FastAPI domain runtime and API-backed UI
bc96e12 Task 5: Implement minimal runnable React UI
1cb43a0 Task 4: Implement controlled UISpec renderer
31d4674 Task 3: Implement UISpec validator and tests
3824b07 Task 2: Formalise UISpec model contract
e289754 Task 1: Establish project foundation
2961344 Initial commit

Task 6 was committed as a55c522 and pushed successfully to origin/main. The final working tree was clean and the local main branch was synchronized with origin/main.

6.15 Task 6 Checkpoint Acceptance Criteria

- Task 6 implementation verified.
- 34/34 Python tests passed.
- 52/52 React tests passed.
- 12/12 JSON Schema checks passed.
- Production build passed.
- FastAPI API smoke tests passed.
- Vite/browser proxy smoke test passed.
- Security/control regression passed.
- No AI/model integration was introduced.
- Task 6A was not started.
- Task 7 was not started.
- Commit a55c522 was created and pushed.
- Working tree is clean and synchronized.

6.16 Human Review Boundary

Task 6 stops at this checkpoint. No Task 7 implementation was started as part of the checkpoint. The project now has a stable Domain Runtime/API seam that can be reviewed before the next increment.

Task 6 — Current Verified Project Status

Area	Status
Business problem	Defined
Project engineering foundation	Complete
Model contract	Complete
JSON Schema	Complete
Deterministic validator	Complete
Controlled React registry	Implemented
Controlled React renderer	Implemented
Runnable React UI	Implemented
FastAPI Domain Runtime	Implemented
API-backed UI	Implemented
Task 6 checkpoint	Complete — a55c522
AI Runtime / Model Gateway	Not yet implemented

Runtime model integration	Not yet implemented
AI evaluation suite	Not yet implemented
Feedback loop	Not yet implemented
Production architecture	Not yet implemented

Task 6 — One-Sentence Walkthrough Summary

Task 6 established and checkpointed the FastAPI Domain Runtime and API-backed React UI, reusing the deterministic UISpec validator and preserving the controlled renderer, while intentionally leaving the AI Runtime for a later task.